

Лабораторна робота №3

Мета: дослідити та реалізувати базові структури: стек, дек, черга, списки.

Завдання:

1. Проаналізувати власний проект щодо використання стеку, деку, черги та списків.
2. Представити приклади реалізації при умові використання базових структур в проекті.
3. Виконати варіанти завдань реалізації стеку, черги, списку (за вибором) або запропонувати власну задачу.

1.1. Стеки

Стеком називається множина деякої змінної кількості даних, над якою виконуються наступні операції:

- Поповнення стеку новими даними;
- Перевірка, яка визначає чи стек пустий;
- Перегляд останніх добавлених даних;
- Знищення останніх добавлених даних.

На основі такого функціонального опису, можна сформулювати логічний опис. Стек – це такий послідовний список із змінної довжиною, включення і виключення елементів з якого виконуються тільки з одного боку списку. Застосовуються і інші назви стеку – магазин, пам'ять що функціонує за принципом LIFO (Last – In – First – Out – „останнім прийшов – першим вийшов”).

Самий „верхній” елемент стеку, тобто останній добавлений і ще не знищений, відіграє особливу роль: саме його можна модифікувати й знищувати. Цей елемент називають вершиною стеку. Іншу частину стеку називають тілом стеку. Тіло стеку, само собою, є стеком: якщо виключити зі стеку його вершину, то тіло перетворюється в стек.

Основні операції над стеком – включення нового елемента (**push** – заштовхувати) і виключення елемента зі стеку (**pop** – вискакувати).

Корисними можуть бути також допоміжні операції:

- визначення поточної кількості елементів в стеку;
- очищення стеку;
- „неруйнуюче” читання елемента з вершини стека, яке може бути реалізоване, як комбінація основних операцій – виключити елемент зі стеку та включити його знову в стек.

При представленні стеку в статичній пам'яті для стеку виділяється пам'ять, як для вектора. В описі цього вектора окрім звичайних для вектора параметрів повинен знаходитися також покажчик стеку – адреса вершини стека. Обмеження даного представлення полягає в тому, що розмір стеку обмежений розмірами вектора.

Покажчик стеку може вказувати або на перший вільний елемент стеку, або на останній записаний в стек елемент. Однаково, який з цих двох

варіантів вибрати, важливо надалі строго дотримуватися його при обробці стеку.

При занесенні елементу в стек елемент записується на місце, яке визначається покажчиком стеку, потім покажчик модифікується так, щоб він вказував на наступний вільний елемент (якщо покажчик вказує на останній записаний елемент, то спочатку модифікується покажчик, а потім проводиться запис елемента). Модифікація покажчика полягає в надбавці до нього або у відніманні від нього одиниці (стек росте у бік збільшення адреси).

Операція виключення елемента полягає в модифікації покажчика стеку (в напрямку, зворотному модифікації при включенні) і вибірці значення, на яке вказує покажчик стеку. Після вибірки комірка, в якій розміщувався вибраний елемент, вважається вільною.

Операція очищення стеку зводиться до запису в покажчик стеку початкового значення – адреси початку виділеної ділянки пам'яті.

Визначення розміру стека зводиться до обчислення різниці покажчиків: покажчика стеку й адреси початку ділянки.

При зв'язному представленні стеку кожен елемент стеку складається із значення і покажчика, який вказує на попередньо занесений у стек елемент. Зв'язне представлення викликає втрату пам'яті, що викликане наявністю покажчика в кожному елементі стеку, і представляє інтерес тільки у випадку, коли важко визначити максимальний розмір стеку.

Отже, для зв'язного представлення стеку потрібно, щоб кожен його елемент описувався структурою, яка поєднує дані і покажчик на наступний елемент.

Для виконання операцій над стеком потрібен один покажчик на вершину стеку. Створення пустого стеку полягатиме у присвоєнні покажчику на вершину нульового значення, що означатиме, що стек пустий.

Послідовність кроків для додавання елементу в стек складається з декількох кроків:

1. Виділити пам'ять під новий елемент стеку;
2. Занесення значення в інформаційне поле;
3. Встановлення зв'язку між ним і „старою” вершиною стеку;
4. Переміщення вершини стеку на новий елемент:.

Вилучення елемента зі стеку також проводять за кілька кроків:

1. Зчитування інформації з інформаційного поля вершини стеку;
2. Встановлення на вершину стеку допоміжного покажчика;
3. Переміщення покажчика вершини стеку на наступний елемент;
4. Звільнення пам'яті, яку займає „стара” вершина стеку.

1.2. Черга

Чергою називається множина змінної кількості даних, над якою можна виконувати наступні операції:

- Поповнення черги новими даними;
- Перевірка, яка визначає чи пуста черга;
- Перегляд перших добавлених даних;

- Знищення самих перших добавлених даних.

На основі такого функціонального опису, можна сформувати логічний опис. Чергою FIFO (First – In – First – Out – „першим прийшов – першим вийшов”). називається такий послідовний список із змінній довжиною, в якому включення елементів виконується тільки з одного боку списку (хвіст черги), а виключення – з другого боку (голова черги).

Основні операції над чергою – ті ж, що і над стеком – включення, виключення, визначення розміру, очищення, „неруйнуюче” читання.

При представленні черги вектором в статичній пам’яті на додаток до звичайних для опису вектора параметрів в ньому повинні знаходитися два покажчики: на голову і на хвіст черги. При включенні елемента в чергу елемент записується за адресою, яка визначається покажчиком на хвіст, після чого цей покажчик збільшується на одиницю. При виключенні елемента з черги вибирається елемент, що адресується покажчиком на голову, після чого цей покажчик зменшується на одиницю.

Очевидно, що з часом покажчик на хвіст при черговому включенні елемента досягне верхньої межі тієї ділянки пам’яті, яка виділена для черги. Проте, якщо операції включення чергувати з операціями виключення елементів, то в початковій частині відведеної під чергу пам’яті є вільне місце. Для того, щоб місця, займані виключеними елементами, могли бути повторно використані, черга замикається в кільце: покажчики (на початок і на кінець), досягнувши кінця виділеної області пам’яті, перемикаються на її початок. Така організація черги в пам’яті називається кільцевою чергою. Можливі, звичайно, і інші варіанти організації: наприклад, всякий раз, коли покажчик кінця досягне верхньої межі пам’яті, зсовувати всі не порожні елементи черги до початку ділянки пам’яті, але як цей, так і інші варіанти вимагають переміщення в пам’яті елементів черги і менш ефективні, ніж кільцева черга.

У початковому стані покажчики на голову і хвіст вказують на початок ділянки пам’яті. Рівність цих двох покажчиків є ознакою порожньої черги. Якщо в процесі роботи з кільцевою чергою кількість операцій включення перевищує кількість операцій виключення, то може виникнути ситуація, в якій покажчик кінця „наздожене” покажчик початку. Це ситуація заповненої черги, але якщо в цій ситуації покажчики порівнюються, ця ситуація буде така ж як при порожній черзі. Для розрізнення цих двох ситуацій до кільцевої черги пред’являється вимога, щоб між покажчиком кінця і покажчиком початку залишався „проміжок” з вільних елементів. Коли цей „проміжок” скорочується до одного елемента, черга вважається заповненою і подальші спроби запису в неї блокуються. Очищення черги зводиться до запису одного і того ж (не обов’язково початкового) значення в обидва покажчики. Визначення розміру полягає в обчисленні різниці покажчиків з урахуванням кільцевої природи черги.

При зв’язному представленні черги кожен елемент черги складається із значення і покажчика, який вказує на попередньо занесений у чергу елемент.

Зв’язне представлення викликає втрату пам’яті, що викликане наявністю покажчика в кожному елементі черги, і представляє інтерес тільки

у випадку, коли важко визначити максимальний розмір черги. Для зв'язного представлення черги потрібно, щоб кожен його елемент описувався структурою, яка поєднує дані і покажчик на наступний елемент.

Для виконання операцій над чергою потрібно два покажчики: на голову і хвіст черги. Створення пустої черги полягатиме у присвоєнні покажчикам на голову і хвіст черги нульових значень, що означатиме, що черга пуста.

Послідовність кроків для додавання елемента в кінець черги складається з декількох кроків:

1. Виділити пам'ять під новий елемент черги;
2. Занесення значення в інформаційне поле;
3. Занесення нульового значення в покажчик;
4. Встановлення зв'язку між ним і останнім елементом черги і новим, враховуючи випадок пустої черги;
5. Переміщення покажчика кінця черги на новий елемент.

Вилучення елемента з черги також проводять за кілька кроків:

1. Зчитування інформації з інформаційного поля голови черги;
2. Встановлення на голову черги допоміжного покажчика;
3. Переміщення покажчика початку черги на наступний елемент;
4. Звільнення пам'яті, яку займав перший елемент черги.

В реальних задачах іноді виникає необхідність у формуванні черг, відмінних від приведених структур. Порядок вибірки елементів з таких черг визначається пріоритетами елементів. Пріоритет в загальному випадку може бути представлений числовим значенням, яке обчислюється або на підставі значень яких-небудь полів елемента, або на підставі зовнішніх чинників. Так попередньо наведені структури стек і черги можна трактувати як пріоритетні черги, в яких пріоритет елемента залежить від часу його включення в структуру. При вибірці елемента всякий раз вибирається елемент з щонайбільшим пріоритетом.

Черги з пріоритетами можуть бути реалізовані на лінійних структурах – в суміжному або зв'язному представленні. Можливі черги з пріоритетним включенням – в яких послідовність елементів черги весь час підтримується впорядкованою, тобто кожний новий елемент включається на те місце в послідовності, яке визначається його пріоритетом, а при виключенні завжди вибирається елемент з голови. Можливі і черги з пріоритетним виключенням – новий елемент включається завжди в кінець черги, а при виключенні в черзі шукається (цей пошук може бути тільки лінійним) елемент з максимальним пріоритетом і після вибірки вилучається з послідовності. І в тому, і в іншому варіанті потрібний пошук, а якщо черга розміщується в статичній пам'яті – ще і переміщення елементів.

1.3. Деки

Дек – особливий вид черги. Дек (deq – double ended queue, тобто черга з двома кінцями) – це такий послідовний список, в якому як включення, так і виключення елементів, може здійснюватися з будь-якого з двох кінців списку. Так само можна сформулювати поняття деку, як стек, в якому включення і виключення елементів може здійснюватися з обох кінців.

Деки рідко зустрічаються у своєму первісному визначенні. Окремий випадок деку – дек з обмеженим входом і дек з обмеженим виходом. Логічна і фізична структури деку аналогічні логічній і фізичній структурі кільцевої черги. Проте, стосовно деку доцільно говорити не про голову і хвіст, а про лівий і правий кінець.

Над деком доступні наступні операції:

- включення елемента праворуч;
- включення елемента ліворуч;
- виключення елемента з права;
- виключення елемента з ліва;
- визначення розміру;
- очищення.

Фізична структура деку в статичній пам'яті ідентична структурі кільцевої черги.

1.4. Лінійні списки

Лінійні списки є узагальненням попередніх структур; вони дозволяють представити множину так, щоб кожний елемент був доступний і при цьому не потрібно було б зачіпати деякі інші.

Списки є досить гнучкою структурою даних, так як їх легко зробити більшими або меншими, і їх елементи доступні для вставки або вилучення в будь-якій позиції списку. Списки також можна об'єднувати або розділяти на менші списки.

Лінійний список – це скінчена послідовність однотипних елементів (вузлів), можливо, з повторенням. Кількість елементів у послідовності називається довжиною списку. Вона в процесі роботи програми може змінюватися. Лінійний список L , що складається з елементів $d_1, d_2, \dots, d_n$, які мають однаковий тип, записують у вигляді $\boxed{d1} \rightarrow \boxed{d2} \rightarrow \boxed{d3} \rightarrow \dots \rightarrow \boxed{dn}$ $L = \langle d_1, d_2, \dots, d_n \rangle$, або зображають графічно.

Важливою властивістю лінійного списку є те, що його елементи можна лінійно впорядкувати у відповідності з їх позицією в списку.

Для формування абстрактного типу даних на основі математичного визначення списку потрібно задати множину операторів, які виконуються над об'єктами типу список. Проте не існує однієї множини операторів, які виконуються над списками, які задовольняють відразу всі можливі застосування.

Найчастіше зі списками доводиться виконувати такі операції:

- Знайти елемент із заданою властивістю;
- Визначити i -й елемент у лінійному списку;
- Внести додатковий елемент до або після вказаного вузла;
- Вилучити певний елемент списку;
- Впорядкувати вузли лінійного списку в певному порядку.

У реальних мовах програмування не існує якої-небудь структури даних для зображення лінійного списку так, щоб усі операції над ним виконувалися в однаковій мірі ефективно. Тому при роботі з лінійними списками важливе

значення має подання лінійних списків, які використовуються в програмі, таким чином, щоб була забезпечена максимальна ефективність і за часом виконання програми, і за обсягом потрібної їй пам'яті.

Лінійний список є послідовність об'єктів. Позиція елемента в списку має інший тип даних, відмінний від типу даних елемента списку, і цей тип залежить від конкретної фізичної реалізації.

Над лінійним списком допустимі наступні операції.

Операція вставки – вставляє елемент в конкретну позицію в списку, переміщуючи елементи від цієї позиції і далі в наступну, більш вищу позицію.

Операція локалізації – повертає позицію об'єкта в списку. Якщо в списку об'єкт зустрічається декілька разів, то повертається позиція першого від початку списку об'єкта. Якщо об'єкта немає в списку, то повертається значення, яке рівне довжині списку, збільшене на одиницю.

Операція вибірки елемента з списку – повертає елемент, який знаходиться в конкретній позиції списку. Результат не визначений, якщо в списку немає такої позиції.

Операція вилучення – вилучає елемент в конкретній позиції зі списку. Результат невизначений, якщо в списку немає вказаної позиції.

Операції вибірки попереднього і наступного елемента – повертають відповідно наступний і попередній елемент списку відносно конкретної позиції в списку.

Функція очистки списку робить список пустим.

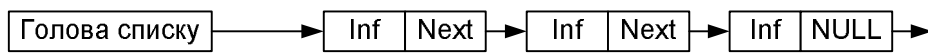
Основні методи зберігання лінійних списків поділяються на методи послідовного і зв'язного зберігання. При виборі способу зберігання в конкретній програмі слід враховувати, які операції і з якою частотою будуть виконуватися над лінійними списками, вартість їх виконання та обсяг потрібної пам'яті для зберігання списку.

Найпростіша форма представлення лінійного списку — це вектор. Визначивши таким чином список можна по чергово звертатися до них в циклі і виконувати необхідні дії. Однак при такому представленні лінійного списку не вдасться уникнути фізичного переміщення елементів, якщо потрібно добавляти нові елементи, або вилучати існуючі. Набагато швидше вилучати елементи можна за допомогою простої схеми чистки пам'яті. Замість вилучення елементів із списку, їх помічають як невикористані.

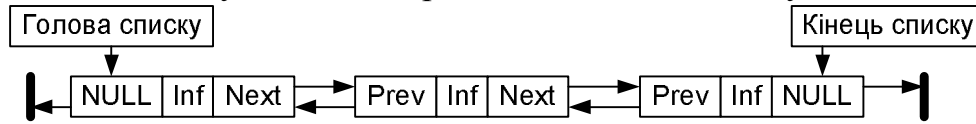
Більш складною організацією при роботі зі списками є розміщення в масиві декількох списків або розміщення списку без прив'язки його початку до першого елемента масиву.

При зв'язному представленні лінійного списку кожен його елемент складається із значення і покажчика, який вказує на наступний елемент у списку.

На наступному рисунку приведена структура однозв'язного списку. Кожний список повинен мати особливий елемент, який називається покажчиком на початок списку, або головою списку.

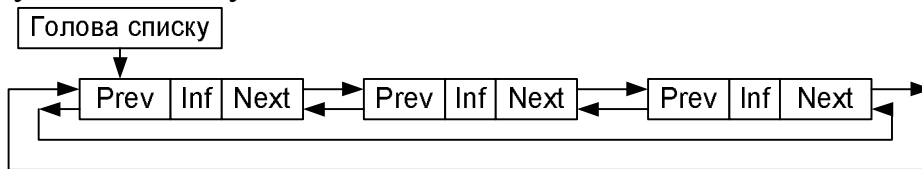


Проте, обробка однозв'язного списку не завжди зручна, оскільки відсутня можливість просування в протилежну сторону. Таку можливість забезпечує двох-зв'язний список, кожний елемент якого містить два показники: на наступний і попередній елементи списку.



Для зручності обробки списку додають ще один особливий елемент – показник кінця списку. Наявність двох показників в кожному елементі ускладнює список і приводить до додаткових витрат пам'яті, але в той же час забезпечує більш ефективне виконання деяких операцій над списком.

Різновидом розглянутих видів лінійних списків є кільцевий список, який може бути організований на основі як однозв'язного, так і двох-зв'язного списків. При цьому в однозв'язному списку показник останнього елемента повинен вказувати на перший елемент; в двох-зв'язному списку в першому і останньому елементах відповідні показники змінюються.

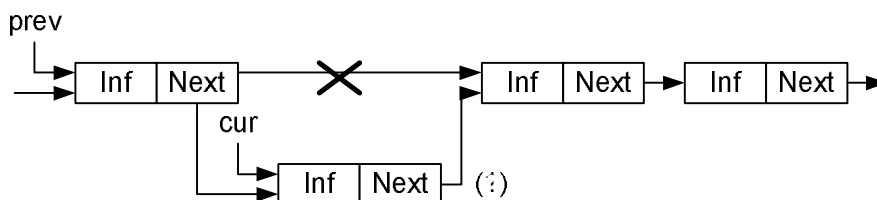


При роботі з такими списками дещо спрощуються деякі процедури, проте, при перегляді такого списку слід приймати деякі запобіжні засоби, щоб не потрапити в нескінченний цикл.

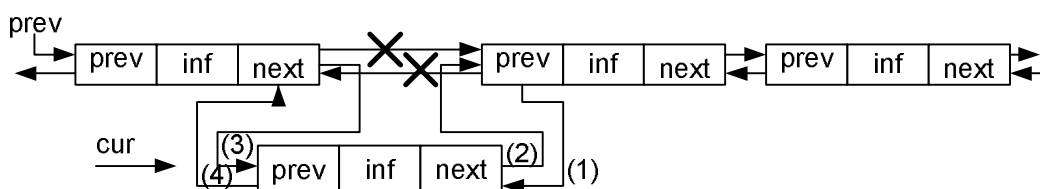
В пам'яті список є сукупністю опису однакових за розміром і форматом структур, які розміщені довільно в деякій ділянці пам'яті і пов'язані одна з одною в лінійно впорядкований ланцюжок за допомогою показників. Структура містить інформаційні поля і поля показників на сусідні елементи списку, причому деякими полями інформаційної частини можуть бути показники на блоки пам'яті з додатковою інформацією, що відноситься до елемента списку.

Розглянемо деякі прості операції над лінійними списками.

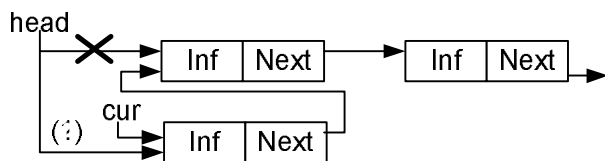
Вставка елемента в середину однозв'язного списку:



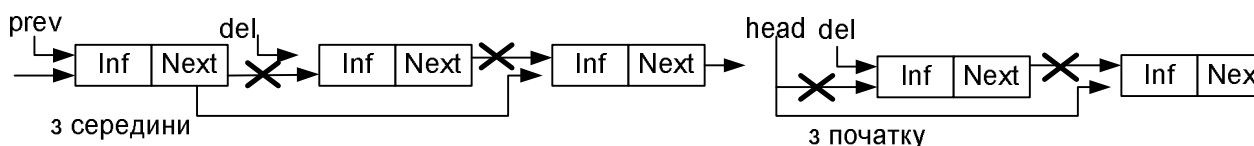
Вставка елемента в двох-зв'язний список:



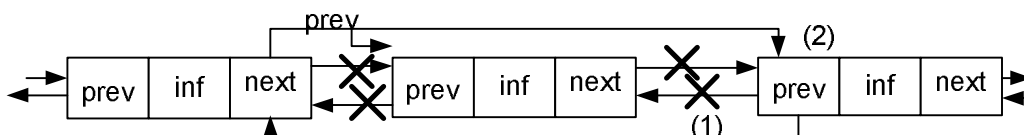
Наведені приклади забезпечують вставку в середину списку, але не можуть бути застосовані для вставки на початок списку. При такій операції повинен модифікуватися показник на початок списку:



Видалення елемента з однозв'язного списку для двох варіантів – з середини і з голови:

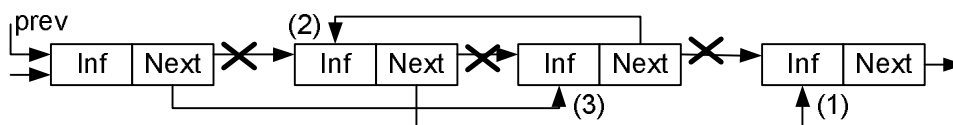


Видалення елемента з двох-зв'язного списку вимагає корекції більшої кількості показників:



Процедура видалення елемента з двох-зв'язного списку виявиться навіть простішою, ніж для однозв'язного, оскільки в ній не потрібно шукати попередній елемент, він вибирається за показником назад.

Змінність динамічних структур даних допускає не тільки зміни розміру структури, але і зміни зв'язків між елементами. Для зв'язних структур зміна зв'язків не вимагає пересилки даних в пам'яті, а тільки зміни показників в елементах зв'язної структури. В якості прикладу приведена перестановка двох сусідніх елементів списку. В алгоритмі перестановки в однозв'язному списку виходили з того, що відома адреса елемента, який передує парі, в якій проводиться перестановка. В приведеному алгоритмі також не враховується випадок перестановки початкових елементів списку.



У процедурі перестановки для двох-зв'язного списку неважко врахувати і перестановку на початку списку.

1.5. Мультисписки

В програмних системах, які обробляють об'єкти складної структури, можуть вирішуватися різні підзадачі, кожна з яких вимагає обробки можливо не всієї множини об'єктів, а лише якоїсь його підмножини.

Для того, щоб при вибірці кожної підмножини не виконувати повний перегляд з відсіванням записів, які до необхідної підмножини не відносяться, в кожний запис включаються додаткові поля посилань, кожне з яких зв'язує в лінійний список елементи відповідної підмножини. В результаті виходить

багато-зв'язковий список або мультисписок, кожний елемент якого може входити одночасно в декілька однозв'язних списків.

До переваг мультисписків крім економії пам'яті (при множині списків інформаційна частина існує в єдиному екземплярі) слід віднести також цілісність даних – в тому сенсі, що всі підзадачі працюють з однією і тією ж версією інформаційної частини і зміни в даних, зроблені однієї підзадачею негайно стають доступними для іншої підзадачі.

Кожна підзадача працює з своєю підмножиною як з лінійним списком, використовуючи для цього певне поле зв'язку. Специфіка мультисписку виявляється тільки в операції виключення елемента із списку. Виключення елемента з якого-небудь одного списку ще не означає необхідності видалення елемента з пам'яті, оскільки елемент може залишатися у складі інших списків. Пам'ять повинна звільнитися тільки у тому випадку, коли елемент вже не входить ні в один з приватних списків мультисписку. Звичайно задача видалення спрощується тим, що один з приватних списків є головним – в нього обов'язково входять всі наявні елементи. Тоді виключення елемента з будь-якого неголовного списку полягає тільки в зміні покажчиків, але не в звільненні пам'яті. Виключення ж з головного списку вимагає не тільки звільнення пам'яті, але і зміни покажчиків як в головному списку, так і у всіх неголовних списках, в які елемент, що видаляється, входив.

Варіант №1.

Реалізувати стек для зберігання і операцій з даними виду:

Ім'я процедури	Кількість параметрів	Параметри (по 2 байти)
----------------	----------------------	------------------------

Забезпечити виконання операцій:

- додавання процедури в стек;
- видалення процедури зі стека;
- видалення стека (вивільнення місця у пам'яті);
- роздрукування вмісту стека;
- підрахунок розміру пам'яті, зайнятої стеком;
- підрахунок кількості процедур, що знаходиться у стеку.

Варіант №2.

Реалізувати стек для зберігання і операцій з даними виду:

Ім'я функції	Значення, що повертається (6 байтів)	Кількість параметрів	Параметри (по 6 байтів)
--------------	--------------------------------------	----------------------	-------------------------

Забезпечити виконання операцій:

- виділення місця в пам'яті під стек;
- додавання функції в стек;
- видалення функції із стека;
- попередження про можливе переповнювання;
- повідомлення про переповнювання;

- роздрукування вмісту стека;
- звільнення пам'яті, виділеної під стек.

Варіант №3.

Реалізувати чергу для зберігання і операцій з даними виду:

Прізвище	Спеціальність	Дата подання заяви	Дата реєстрації на біржі праці
----------	---------------	--------------------	--------------------------------

Забезпечити виконання операцій:

- додавання елемента в чергу;
- видалення елемента з черги;
- визначення розміру черги;
- роздрукування черги;
- знаходження в черзі елемента за прізвищем з вказівкою черговості;
- формування списку по заданій спеціальності.

Варіант №4.

Реалізувати чергу для зберігання і операцій з даними виду:

Ім'я програми	Мова програмування	Розмір пам'яті, що потребується	Час виконання
---------------	--------------------	---------------------------------	---------------

Забезпечити виконання операцій:

- виділення місця в пам'яті під чергу;
- додавання елемента в чергу;
- видалення елемента з черги;
- попередження про можливе переповнювання;
- повідомлення про переповнювання;
- визначення часу виконання всіх програм;
- роздрукування черги;
- звільнення пам'яті, виділеної під чергу.

Варіант №5.

Реалізувати кільцеву чергу (циклічний список) для зберігання і операцій з даними виду:

Найменування пристрою	Рік випуску	Вартість
-----------------------	-------------	----------

Забезпечити виконання операцій:

- додавання елемента в чергу;
- видалення елемента з черги в порядку запису;
- визначення позиції елемента у черзі;
- видалення всіх елементів із заданим полем;
- роздрукування черги.

Варіант №6.

Реалізувати кільцеву чергу для зберігання і операцій з даними виду:

Ім'я програми	Час виконання
---------------	---------------

Забезпечити виконання операцій:

- формування черги;
- автоматична прокрутка черги з виділенням "кванта часу центрального процесора";
- додавання елементів в чергу;
- автоматичне виведення елементів з черги по використанню часу процесора (видати повідомлення);
- видача довідки про кількість елементів в черзі;
- роздрукування черги.

Варіант №7.

Реалізувати однозв'язний список для зберігання і операцій з даними виду:

Найменування мікросхеми	Вартість	Кількість
-------------------------	----------	-----------

Забезпечити виконання операцій:

- додавання елементу в неврегульований список;
- видалення елементу із заданим полем з неврегульованого списку;
- додавання елементу після вказаного;
- видалення всіх елементів із заданим полем;
- роздрукування списку.

Варіант №8.

Реалізувати однозв'язний список для зберігання і операцій з даними виду:

Ім'я змінної	Тип	Кількість байтів
--------------	-----	------------------

Забезпечити виконання операцій:

- сортування списку за полім "Ім'я";
- додавання елементу у впорядкований список;
- видалення елементу із списку;
- зміну полів "Тип" і "Число байтів" елементу, визначуваних полем "Ім'я змінної";
- роздрукування списку.

Варіант №9.

Реалізувати однозв'язний список для зберігання і операцій з даними виду:

Країна	Місто	Кількість мешканців
--------	-------	---------------------

Забезпечити виконання операцій:

- додавання елементу в список;
- видалення елементу із списку;
- розділення списку на два за ознакою: "Країна", що має міста з числом мешканців $\geq K$ і $< K$;
- підрахунок "Числа мешканців" у всіх містах заданої країни;

- роздрукування списку.

Варіант №10.

Реалізувати однозв'язний список для зберігання і операцій з даними виду:

Об'єкт	Кількість зовнішніх зв'язків	Зовнішні зв'язки
--------	------------------------------	------------------

Забезпечити виконання операцій:

- установка максимального розміру списку;
- контроль переповнювання списку;
- додавання об'єкту в список;
- видалення об'єкту із списку;
- роздрукування списку;
- знаходження вказаного об'єкту в списку.

Варіант №11.

Реалізувати лінійний двозв'язний список для зберігання і операцій з даними виду:

Прізвище	Рік вступу	Середній бал
----------	------------	--------------

Забезпечити виконання операцій:

- додавання елемента в неврегульований список;
- сортування списку за полем "Прізвище";
- додавання елемента у впорядкований список;
- видалення елемента із списку;
- роздрукування списку;
- розділення списку на два за значенням поля "Середній бал".

Варіант №12.

Реалізувати двозв'язний список з розгалуженнями для зберігання і операцій з даними виду:

Ім'я	Послідовність чисел
------	---------------------

Забезпечити виконання операцій:

- додавання елементів в список;
- пошук і роздрукування елемента в списку;
- видалення елемента зі списку;
- підрахунок елементів у списку.
- роздрукування списку.

Примітка. Послідовність чисел може містити від 1 до N чисел. Для послідовностей завдовжки більш K ($K < N$) організувати гілки.

Варіант №13.

Реалізувати двозв'язний список з розгалуженнями для зберігання і операцій з даними виду:

Ім'я	Послідовність символів (довідник)
------	-----------------------------------

Забезпечити виконання операцій:

- додавання елементів в невпорядкований список;
- сортування списку за полем ім'я;
- додавання елементу в відсортований список;
- видалення елементу зі списку.
- роздрукування групи елементів з довжиною "Послідовність символів" $\geq K$;
- редагування послідовності символів (заміна на нову).

Примітка. Послідовність чисел може містити від 1 до N чисел. Для послідовностей завдовжки більш K ($K < N$) організувати гілки.

Варіант №14.

Реалізувати двозв'язний список з розгалуженнями для зберігання і операцій з даними виду:

Ім'я процедури	Кількість параметрів	Значення параметрів (по 4 байти)
----------------	----------------------	----------------------------------

Забезпечити виконання операцій:

- додавання елементів в список;
- видалення елементу з заданим ім'ям зі списку;
- визначення кількості процедур у списку;
- визначення зайнятого місця у пам'яті під список;
- роздрукування усього списку;
- роздрукування списку процедур без параметрів.

Варіант №15.

Реалізувати двозв'язний список для зберігання і операцій з даними виду:

Найменування моделі	Дата виготовлення	Кількість
---------------------	-------------------	-----------

У перший підсписок входять усі записи. В другий - лише ті записи, де поле "Кількість" $< K$.

Забезпечити виконання операцій:

- додавання нового елементу в список;
- пошук елементу за полем кількість;
- роздрукування підписків;
- корегування значення поля "Кількість" вибраного елементу.

Варіант №16.

Реалізувати двозв'язний список для зберігання і операцій з даними виду:

Прізвище	Країна	Рік вступу	Вартість навчання
----------	--------	------------	-------------------

У перший підсписок входять усі записи. В другий - лише ті записи, де поле "Країна" не дорівнює "Україна".

Забезпечити виконання операцій:

- додавання нового елементу в невідсортований список;
- відсортування списку за полем "Прізвище";
- додавання елемента в відсортований список;
- коректування поля "Країна" обраного елемента;
- роздрукування підписків;
- роздрукування переліку країн.

Варіант №17.

Реалізувати тривимірний список для зберігання і операцій з даними виду:

Товар	Вартість	Кількість	Дата постачання
-------	----------	-----------	-----------------

У перший підсписок входять усі записи. В другий - лише ті, у яких "Вартість" більша ніж С. В третій - лише ті, дата постачання яких більша, ніж D.

Забезпечити виконання операцій:

- додавання нового елемента в список;
- роздрукування списків;
- видалення елемента з заданим полем;
- знаходження сумарної вартості товару;
- зміна критерію С.

Варіант №18.

Реалізувати тривимірний список для зберігання і операцій з даними виду:

Відділ	Тип ПК	Кількість	Наявність мережі
--------	--------	-----------	------------------

У перший підсписок входять усі записи. В другий - лише ті, у яких встановлені найсучасніші ПК. В третій - відділи, де є комп'ютерна мережа.

Забезпечити виконання операцій:

- додавання нового елемента в список;
- дооснащення відділу технікою;
- списання старої техніки;
- встановлення у відділі мережевого обладнання;
- роздрукування списків;
- визначення відділів з найкращим технічним забезпеченням.

Варіант №19.

Реалізувати тривимірний список для зберігання і операцій з даними виду:

Прі звище	Рік народження	Освіта	Стаж
--------------	-------------------	--------	------

У перший підписок входять усі записи. В другий - лише ті, у яких "Рік народження" менше, ніж **G**. В третій - лише ті, у яких стаж більше **n** років. **G**, **n**- визначаються самостійно.

Забезпечити виконання операцій:

- додавання нового елемента у невідсортований список;
- сортування списку за полем "Прізвище";
- додавання нового елемента у відсортований список;
- роздрукування третього підписку;
- видалення елемента з першого списку;
- визначення кількості осіб з вищою освітою.

Звіт:

Титульний лист.

Умова задачі.

Алгоритм

Лістинг

Детальні коментарі в лістингу програм и можуть бути алгоритмом.

Висновки